VeriSAGE: Extended Report
Proofs, Definitions, and Additional Results

Hoang Nhat Khang Nguyen, Minh Hieu Le, Huynh-Gia-Huy Pham,
Phat T. Tran-Truong, and Xuan-Bach Le

Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam

This report accompanies the ICSOC 2026 Research Project Exhibition paper on VeriSAGE (the "main paper"). It develops the proofs and ordering semantics in linear temporal logic on finite traces (LTL_f), then extends the service-level agreement (SLA) bounds and evaluation. We restate proved results in full so that the arguments can be read independently. Our public artifact repository holds code, prompts, and result files: `https://github.com/lxbachcs/VeriSAGE-artifact`. Reported measurements come from the artifact's `results/` folder.

Design versus prototype. We prove the guarantees for the design in main Sect. 3.1. The prototype instantiates it with propositional pre/postconditions, Boolean policy guards, and sequential plans (main Sect. 3.2). Each check decides its implemented predicate, preserving the soundness argument within this contract language.

1 Proofs

Throughout, $G = (\mathsf{in}_G, \mathsf{out}_G, \mathsf{post}_G, \Phi, B, \delta)$ specifies the initial and target schemas, final condition, ordering rules, latency budget, and tolerated violation probability. We write $\mathrm{Adm}_G(\mathcal{C}) = 1$ if and only if composition $\mathcal{C}$ passes C1 to C5: schemas, pre/postconditions, ordering, data flow, and access/latency (main Table 2). We use the contracted service transition system (CSTS) of main Definition 2, whose states record available fields $R(q)$ and established facts $\sigma(q)$. A *typed witness* χ names a failing check and a minimal piece of evidence (a call with an unbound input, an unmet precondition, a violating trace prefix, a source-to-sink path, a denying guard, or an SLA bound above δ). Its *class* $[\chi]$ is the set of plans up to a chosen length bound $L_{\max}$ that exhibit the same evidence. CompileRepair adds $[\chi]$ to the refuted set. We write U for the plans of length at most $L_{\max}$ outside every refuted class. We allow at most T proposal attempts and write $\mathcal{C}^\star$ for a certified output. The key distinction is between finding a plan and checking it: only the latter determines admission.

Theorem 1 (Execution soundness, main Thm. 1). *If VeriSAGE returns $\mathcal{C}^\star$, then $\mathrm{Adm}_G(\mathcal{C}^\star) = 1$, whatever the planner proposes.*

Proof. The loop (main Fig. 2) returns a plan only when every check succeeds, so it suffices that each check succeeds only when its predicate holds. **C1**: a call

is accepted only when a binding $m : \mathsf{in}(s) \to R(q)$ to type-compatible available fields exists (main Def. 2). Subtyping of JSON-Schema records is decidable, so admitted plans call only catalogue operations with well-typed inputs. **C2**: accepted only when a satisfiability modulo theories (SMT) solver proves $\mathsf{pre}(s_i)$ in q_{i-1} and post_G in the final state. SMT decision procedures for the quantifier-free theory are sound [9]. **C3**: accepted iff, for each $\varphi \in \Phi$, the automaton for $\neg\varphi$ rejects the plan's trace [3,12]. By Sect. 2, one linearisation decides Φ for all linearisations. **C4**: accepted iff reachability on the finite acyclic data-flow graph built from the $\mathsf{dep}(s_i)$ finds no path from a field to a sink cleared below its level that avoids every declassifier. Because dep over-approximates the true flow, this is sound for explicit flows [4]. **C5**: accepted iff every guard evaluates to `allow` [2] and the bound of Theorem 2 is at most δ. Hence $\mathrm{Adm}_G(\mathcal{C}^\star) = 1$. The argument never refers to the planner. □

Proposition 1 (Repair never excludes a good plan, main Prop. 1). *Every refuted class contains only inadmissible plans, so U always contains every admissible plan. If a planner that enumerates all of U finds no plan, no admissible plan of length at most $L_{\max}$ exists, and* INFEASIBLE *is correct. The loop halts within T rounds.*

Proof. Every $\mathcal{C} \in [\chi]$ exhibits the evidence of χ, so the check named by χ fails on $\mathcal{C}$ and $\mathrm{Adm}_G(\mathcal{C}) = 0$. Refuting $[\chi]$ therefore removes only inadmissible plans. If U is empty, the admissible plans, all of which lie in U, do not exist. Halting: the loop has at most T rounds and each check terminates. □

Proposition 2 (Repair progress). *The number of distinct witness classes is finite, so at most $N = |\{[\chi]\}|$ rounds can fail with a new class. If the planner proposes only plans in U, every failing round strictly shrinks U, and the loop ends within $\min(T, N)$ rounds with a certified plan,* INFEASIBLE *(for a planner that enumerates U), or* ABSTAIN.

Proof. Witnesses are built from finitely many operations, fields, propositions, and prefixes of length at most $L_{\max}$, so there are finitely many classes, and a refuted class is never re-admitted. A proposed plan from U that fails lies in the new class $[\chi]$, which is then removed from U. □

In the prototype's LLM loop the planner may propose a plan outside U. A plan in a refuted class simply fails the same check again (main Sect. 3.2). Soundness and the INFEASIBLE verdict do not depend on this restriction, which is needed only for the progress bound.

Corollary 1 (Relative completeness). *With unbounded budget and a planner that eventually proposes every plan in U, VeriSAGE returns a certified plan whenever an admissible plan of length at most $L_{\max}$ exists, and* INFEASIBLE *otherwise. Missing an admissible plan is thus only possible when the planner never proposes it.*

Proof. U always contains every admissible plan. If one exists, the planner eventually proposes it and it is certified. Otherwise, every proposal is refuted until U is empty, and INFEASIBLE is correct. □

Theorem 2 (Distribution-free SLA bound, main Thm. 2). *Let a sequential plan have n services with independent latencies $\ell_i \in [0, u_i]$, where u_i bounds service i's delay, and total mean $\mu = \sum_i \mathbb{E}[\ell_i]$. Here $\mathbb{E}$ denotes expectation and $\mathbb{P}$ probability. For $B \geq \mu$, $\mathbb{P}[\sum_i \ell_i > B] \leq \exp\big(-2(B-\mu)^2/\sum_i u_i^2\big)$, so the plan is certified when $B - \mu \geq \sqrt{\frac{1}{2}\ln(1/\delta)\sum_i u_i^2}$.*

Proof. Hoeffding's inequality [6] for independent bounded variables gives $\mathbb{P}[S - \mu > t] \leq \exp(-2t^2/\sum_i u_i^2)$ for $S = \sum_i \ell_i$ and $t \geq 0$. Take $t = B - \mu$ and require the right-hand side to be at most δ. □

Cost obeys the same bound. For availability, a plan fails if any of its n services is down, so by the union bound $\mathbb{P}[\text{some service down}] \leq \sum_i (1 - a_i)$, where a_i is service i's availability probability.

Proposition 3 (Efficient checking). *For a fixed formula set Φ, checking a given plan of length k with data-flow graph of E edges takes time polynomial in k and E plus k calls to the C2 decision procedure. The exponential dependence is only in the size of the formulas in Φ.*

Proof. C1 is one binding/subtyping check per call ($O(k)$ schema checks). C2 is k solver calls. C3 runs the single trace of length $k{+}1$ through each automaton for $\neg\varphi$, whose size is $2^{O(|\varphi|)}$, in time linear in k. Because the planner supplies the trace, we avoid model checking's PSPACE cost. C4 is one $O(E)$ reachability pass. C5 is k guard evaluations plus an $O(n)$ arithmetic bound. □

2 C3 Ordering Rules: Semantics and Trace Closure

LTL_f semantics. We first make the ordering language explicit. Let AP be the set of atomic propositions (action and state markers). A plan induces a finite word $\pi = \pi_0\pi_1 \cdots \pi_{n-1} \in (2^{AP})^*$ of length n, where each π_i is the subset of markers true at position i. Formulas are $\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathbf{X}\varphi \mid \varphi\,\mathbf{U}\,\varphi \mid \mathbf{Y}\varphi \mid \varphi\,\mathbf{S}\,\varphi$ with $p \in AP$. The operators $\mathbf{X}$, $\mathbf{U}$, $\mathbf{Y}$, and $\mathbf{S}$ mean next, until, previous, and since. We write $\pi, i \models \varphi$ when formula φ holds at position $0 \leq i < n$, with the following semantics:

$$
\begin{array}{ll}
\pi, i \models p & \text{iff } p \in \pi_i, \\
\pi, i \models \neg\varphi & \text{iff } \pi, i \not\models \varphi, \\
\pi, i \models \varphi_1 \wedge \varphi_2 & \text{iff } \pi, i \models \varphi_1 \text{ and } \pi, i \models \varphi_2, \\
\pi, i \models \mathbf{X}\varphi & \text{iff } i{+}1 < n \text{ and } \pi, i{+}1 \models \varphi, \\
\pi, i \models \varphi_1\,\mathbf{U}\,\varphi_2 & \text{iff } \exists j\ (i \leq j < n):\ \pi, j \models \varphi_2 \text{ and } \forall k\ (i \leq k < j)\ \pi, k \models \varphi_1, \\
\pi, i \models \mathbf{Y}\varphi & \text{iff } i > 0 \text{ and } \pi, i{-}1 \models \varphi, \\
\pi, i \models \varphi_1\,\mathbf{S}\,\varphi_2 & \text{iff } \exists j\ (0 \leq j \leq i):\ \pi, j \models \varphi_2 \text{ and } \forall k\ (j < k \leq i)\ \pi, k \models \varphi_1.
\end{array}
$$

Derived operators: *eventually* $\mathbf{F}\varphi \equiv \mathit{true}\,\mathbf{U}\,\varphi$, *always* $\mathbf{G}\varphi \equiv \neg\mathbf{F}\neg\varphi$, *once* $\mathbf{O}\varphi \equiv \mathit{true}\,\mathbf{S}\,\varphi$. $\mathbf{X}$ is the strong next (false at the last position). A plan satisfies φ iff $\pi, 0 \models \varphi$, and C3 accepts $\mathcal{C}$ iff $\pi(\mathcal{C}), 0 \models \bigwedge_{\varphi\in\Phi}\varphi$. The precedence pattern $\mathbf{G}(b \to \mathbf{O}\,a)$ reads "every b is preceded by some a". The running example's rule $\mathbf{G}(\texttt{charge} \to \mathbf{O}\,\texttt{fraud})$ is of this kind.

Pairwise safety fragment. For directed acyclic graph (DAG) plans, C3 admits only the patterns precedence $\mathbf{G}(b \to \mathbf{O}\,a)$, response, and absence, and their conjunctions. Each φ names an action set A_φ, which fixes the independence relation $I_\Phi = \{(a,b) : \text{no } \varphi \text{ has } \{a,b\} \subseteq A_\varphi\}$. C3 rejects a DAG plan that leaves a dependent pair (named together by some φ, e.g. `charge` and `fraud`) unordered. Remaining concurrency is between independent actions only. Commuting two adjacent independent actions does not change the truth of any formula in the fragment, so all linearisations of the plan agree on Φ and checking one is sound. For sequential plans the argument is vacuous.

Example. With $\varphi_1 = \mathbf{G}(\texttt{charge} \to \mathbf{O}\,\texttt{auth})$ and $\varphi_2 = \mathbf{G}(\texttt{charge} \to \mathbf{O}\,\texttt{kyc})$, the pair $(\texttt{auth}, \texttt{kyc})$ is independent, and `auth,kyc,charge` and `kyc,auth,charge` agree on Φ. Adding $\varphi_3 = \mathbf{G}(\texttt{kyc} \to \mathbf{O}\,\texttt{auth})$ makes the pair dependent: more rules can only remove independent pairs. A formula such as $\mathbf{F}(\texttt{auth} \wedge \mathbf{X}\,\texttt{kyc})$ is excluded, because commuting an action between `auth` and `kyc` changes its truth.

3 Extended SLA Bounds

DAG plans. For a DAG, completion waits for its slowest source-to-sink path. Let Π be the set of these paths and μ_p the mean latency of path p. Total latency is $\max_{p\in\Pi}\sum_{i\in p}\ell_i$, and C5 uses the union bound $\sum_{p\in\Pi}\exp\big(-2(B-\mu_p)^2/\sum_{i\in p}u_i^2\big)$, which reduces to Theorem 2 when $|\Pi| = 1$. To explore the cost of considering every path, we also test a heuristic that sums only the k highest-mean paths. Certifying the full DAG with this heuristic requires a separate bound on the omitted risk.

On random layered DAGs (40,000 Monte-Carlo samples per configuration), the full union bound needs 2.56 to 2.62× the true slack as $|\Pi|$ ranges from 1 to 27. In these tests, per-path Hoeffding slack accounts for the conservatism, with little change as the number of paths grows. Keeping only the highest-mean path met the risk target in all tested configurations, requiring 2.13× the true slack at $|\Pi| = 27$. Retaining the top k paths also met the target in all tested configurations with tied means. A criterion that keeps each path whose tail contribution exceeds a fraction of δ selects k between 6 and 8, at 2.3 to 2.8× conservatism. We analyse these DAG bounds offline, while the prototype verifies sequential plans.

Variance-aware bounds. Hoeffding's bound can be replaced by the still distribution-free Bernstein bound $\exp(-t^2/(2V + \frac{2}{3}bt))$ or Bennett bound

$\exp(-\frac{V}{b^2}h(\frac{bt}{V}))$, where $t = B - \mu$ is the slack, σ_i^2 bounds service i's latency variance, $V = \sum_i \sigma_i^2$ bounds their total variance under independence, $b = \max_i(u_i - \mathbb{E}\ell_i)$ and $h(x) = (1+x)\ln(1+x) - x$. Re-running the 32,000-plan calibration at $\delta = 0.05$, Bennett needs 22.5% less slack than Hoeffding under uniform latencies and 36.1% less under Beta(2,2), up to 4.85× less when the variance is 1% of its maximum. All three bounds met the risk target in every tested configuration. Under the extremal two-point law (latencies concentrated at the range endpoints) Hoeffding is tighter (Bennett needs 20.7% more slack), so taking the minimum of the two is never worse. With draws from the Quality of Web Service (QWS) response-time dataset, Bennett needs 53.7% less slack. The deployed C5 uses Hoeffding.

Correlated latencies. The bounds above assume independence. A scan of the two catalogues finds 11.1% (payment) and 9.5% (regulated) of critical-path pairs plausibly correlated through a shared input or producer. We compare two alternatives. The one-sided Chebyshev (Cantelli) bound $\mathbb{P}[S - \mu \geq t] \leq V^*/(V^* + t^2)$ with worst-case variance $V^* = (\sum_i \sigma_i)^2$ holds for any dependence between delays, where σ_i bounds their standard deviations. We also evaluate the comonotone Fréchet quantile heuristic $\sum_i F_i^{-1}(1-\delta)$, where F_i^{-1} is service i's latency quantile function. The latter is an empirical comparator here, not a universal dependence-free certificate. We couple light-tailed delays using a Gaussian-copula model with correlation parameter ρ. At $\rho \geq 0.9$, the independent Hoeffding bound fails: violations reach 0.053 to 0.068 against $\delta = 0.05$. Cantelli meets the target at every tested ρ (violation 0), whereas Fréchet meets it up to $\rho = 0.9$ and reaches 0.050 at $\rho = 1$. Cantelli costs about 5× the Hoeffding slack at maximal variance but only 1.1 to 1.6× when latencies concentrate. Fréchet costs a flat 1.04×. With latencies drawn from the real QWS marginals, Hoeffding met the target at every tested ρ up to 1 (worst violation 2.7%), and the Cantelli fallback costs 0.8 to 1.5× the Hoeffding slack. Production traces document such shared-back-end correlation [7,10]. A split-conformal certificate [1] could remove the independence assumption without the worst-case price, a possibility we leave to future work.

4 Additional Evaluation Detail

The Synth-TSR generator. Each task is a payment plan: a chain of 0 to 5 filler services followed by `readCard`, `authenticate`, `tokenize`, `chargeCard`, `emailReceipt`, and `analyticsSink`, with per-service latency support drawn from $[20, 60)$, two ordering rules (authenticate before charge and charge eventually followed by receipt), and a budget that admits the clean plan. With probability 1/2 the task is left clean (label admissible). Otherwise, we inject one violation class. For C1, we drop `readCard`. For C2, we drop `authenticate` and add a token producer so only the precondition fails. C3 swaps `chargeCard` with `authenticate`, whereas C4 drops `tokenize` and rewires the raw card number into charge and sink. C5-policy makes the charge guard deny, and C5-SLA cuts

Table 1. Certified success (%) of agentic baselines on the stratified 100-task suite. Local planner: mean over 3 seeds. †hosted planner, one seed. Best-of-N selects the most repeated of N proposals. CRITIC uses schema feedback. Language Agent Tree Search (LATS) uses LLM-scored candidates.

	qwen2.5:7b			gpt-4o-mini†		
Method	All	Easy	Hard	All	Easy	Hard
Accept-first (no verifier)	33.3	65.3	1.3	70.0	100.0	40.0
Best-of-N, self-consistency [13]	33.0	65.3	0.7	70.0	100.0	40.0
Self-Refine [8]	18.3	36.7	0.0	29.0	36.0	22.0
LATS tree search [15]	33.0	64.0	2.0	69.0	94.0	44.0
CRITIC, schema tool [5]	51.7	100.0	3.3	70.0	100.0	40.0
Blind resampling (verifier-gated)	54.3	100.0	8.7	86.0	100.0	72.0
VeriSAGE	**55.3**	**100.0**	**10.7**	**87.0**	**100.0**	**74.0**

the budget to 35%. The 900-task suite has 454 clean and 446 injected tasks. Because the C3 injection also leaves `chargeCard` without its token and precondition, C1 and C2 already reject those tasks, which is why the cumulative catch rate in main Table 2 does not rise at C3. A future generator will inject ordering violations that leave inputs and preconditions intact.

Reduced verifiers. On the 446 injected tasks, C1 catches 33.6% and C1+C2 catches 51.1%. Adding C3 leaves this at 51.1%, adding C4 raises it to 68.4%, and the full set reaches 100%. A schema-plus-precondition gate, the guarantee a typed-tool or specification-consistency checker such as Clover [11] provides, misses about half of the violations.

Agentic baselines. Table 1 expands the comparison in main Sect. 4. Although the task suite is the same as in main Table 5(a), this experiment uses a separate run and protocol. VeriSAGE therefore scores 55.3% overall for `qwen2.5:7b`, compared with 61.3% there. Only BLIND and VeriSAGE consult the full verifier during search. The other baselines are scored by it at the end, with our CRITIC adaptation receiving schema feedback during generation. LATS uses language-model scores to search candidates, with 22 calls for easy tasks and 46 for hard tasks. We compare these methods under their stated budgets, with unequal total model-call counts.

Read Table 1 within each planner, comparing its easy and hard columns. CRITIC's schema feedback brings qwen2.5:7b to 100% on easy tasks but only 3.3% on hard tasks. The richer contracts in the hard catalogue expose failures that input checking alone cannot resolve. Consulting the full verifier during blind resampling raises hard-task success to 8.7% locally and 72.0% with the hosted planner. Witness feedback adds 2.0 percentage points for each, reaching 10.7% and 74.0%. Because easy and hard tasks are equally numerous and easy-task success is already 100%, this becomes a 1.0-point overall gain. In this experiment, access to the full verifier accounts for more of the observed gain than

repair guidance. The remaining local hard-task failures also show that accurate rejection does not ensure that the planner finds an admissible alternative. Unequal call budgets and the hosted planner's single seed limit broader efficiency or significance claims.

Verification timing. At $k = 200$ services and $|\Phi| = 10$, the structural checks cost C1/C2/C3/C4/C5 = 12.3/7.4/18.4/19.8/8.7 μs (66.6 μs in total), and cost grows by 0.58 μs per added formula ($k = 50$, $|\Phi| \leq 40$, $R^2 = 0.999$). With C2 discharged by Z3 4.15 on quantifier-free linear integer arithmetic with equality and uninterpreted functions (QF-LIA+EUF) (0.30 to 1.11 ms per service depending on formula size), C2 accounts for over 99% of the time and the other checks cost 0.28 μs per service. Median end-to-end verification is 4.3 to 9.1 ms for 13 services and 33 to 70 ms for 100 services (light to heavy formula mixes).

Dependency over-approximation (C4). When an operation declares no flows, we treat it as *opaque*, allowing every input to influence every output. This preserves C4 soundness at the cost of false alarms. Marking 25 to 100% of operations opaque adds 55 to 88% spurious dependency edges. On 20,000 plans with known true flow, the false-positive rate (safe plans rejected) rises from 16.8% to 52.7%. We can reduce it to 5.3 to 20.3% by preferring operations with declared flows during retrieval. In these tests, keeping an opaque edge only between type-compatible fields retains 100% recall, raises edge precision from 24.5% to 72.2%, and lowers plan-level false positives from 75.1% to 36.7%. An optional layer of dynamic witnesses reaches 100% precision and 8.5% false positives, but observed executions cannot establish every possible flow. We retain the conservative all-inputs-to-all-outputs rule as the sound default.

SLA tightness. Against the extremal two-point latency law the Hoeffding bound is nearly tight. Under uniform latencies the observed violation stays below 10^{-3} up to $\delta = 0.20$. On 8,000 plans at $\delta = 0.05$: with uniform latencies 88.5% of plans meet the budget but C5 admits 59.1%, so 33.3% of compliant plans are rejected and C5 demands 2.57× the true headroom. With two-point latencies, 73.8% meet it, 58.2% are admitted, 21.2% of compliant plans are rejected (1.47×). Shrinking an over-budget plan until C5 passes takes 2.61 service removals on average, against 0.46 with an exact oracle (3,000 tasks).

Feedback without an LLM. To isolate the information in a witness from an LLM's ability to use it, we give a deterministic planner only feedback on 4,000 tasks with 1 to 3 injected defects, at $T = 6$. With typed constraints, it certifies 100% in 2.20 rounds on average. Naming only the failed checks lowers success to 75.9% and raises the average to 3.40 rounds (92.1%, 76.7%, 57.5% success for 1, 2, 3 defects). Without feedback, it certifies 0%. On 700 plans violating 2 to 4 classes ($T = 8$), repairing in the fixed order C1, C2, C3, C4, policy, SLA certifies 39.4% with no re-entered class. Fixing whichever check fires first certifies 14.0% and re-enters already-fixed classes 3,896 times.

Wrong contracts. We inject contract errors with probability p, using 1,500 tasks per setting. Restrictive errors include a missing declassifier, a spurious ordering rule, an over-tagged field, and a reduced budget. They cause only false rejections: 3.1, 6.8, 12.1 and 19.0% at $p = 0.05, 0.1, 0.2, 0.3$. Deleting the clause that guards an injected violation has the opposite effect, admitting the violating plan in 2.6, 5.3, 9.6 and 14.6% of cases. Those verdicts conform to the erroneous contract. The rates remain below p because structural C1/C2 violations have no single clause to delete. At $p = 0$, both rates are 0.

A classical QoS selector. An exact latency-minimising selector in the style of QoS-aware composition [14], run over the regulated catalogue with real QWS latencies, finds a plan with mean latency 1,648 ms against 2,666 ms for VeriSAGE's certified plan. It does so by using `quickCharge`, which skips authentication, so the plan fails C3 ("`'charge' occurs before 'authenticate'`"). A genetic-algorithm variant never produced a certifiable plan. The 1,018 ms difference is the cost of admissibility, not overhead on a comparable admissible plan.

Infeasibility with a bounded planner. A best-first forward planner over the payment catalogue (10 services) that enumerates all plans up to a length bound finds the single admissible plan of length ≤ 8 in 1.42 s and returns INFEASIBLE on four unsatisfiable variants (no declassifier, no authentication precondition, impossible budget, unproducible goal) in 0.05 to 1.58 s. Exhaustive search confirms each verdict.

Checking the running example. Main Fig. 1 was checked by running the prototype verifier on both plans with the regulated catalogue and five goal seeds. For every seed the output was:

```
PLAN 1  verify_all -> ["[C2] chargeCard precondition ['fraud_ok'] unmet",
                       "[C3] 'charge' occurs before required 'fraud'",
                       "[C4] 'ssn' reaches sink 'auditLog' un-declassified"]
PLAN 2  verify -> ok=True
        verify_all -> []
```

We constructed these inputs to demonstrate verifier behaviour. Per-task outcomes from the separate logged LLM runs are in `results/raw/`.

Planned evaluation. The benchmark of work package WP2 (main Table 3) will compare VeriSAGE with frontier agents and classical QoS selectors on tasks built from public OpenAPI specifications. Significance tests (per-task McNemar or Wilcoxon with Holm-Bonferroni correction) will be computed only on measured paired outcomes. This planned benchmark extends the completed experiments reported here and in the main paper.

References

1. Angelopoulos, A.N., Bates, S.: Conformal prediction: A gentle introduction. Found. Trends Mach. Learn. **16**(4), 494–591 (2023)

2. Cloud Native Computing Foundation: Open policy agent: Policy-based control and the Rego language (2023), https://www.openpolicyagent.org
3. De Giacomo, G., Vardi, M.Y.: Linear temporal logic and linear dynamic logic on finite traces. In: Proc. IJCAI. pp. 854–859 (2013)
4. Denning, D.E.: A lattice model of secure information flow. Commun. ACM **19**(5), 236–243 (1976)
5. Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., Chen, W.: CRITIC: Large language models can self-correct with tool-interactive critiquing. In: Proc. ICLR (2024)
6. Hoeffding, W.: Probability inequalities for sums of bounded random variables. J. Am. Stat. Assoc. **58**(301), 13–30 (1963)
7. Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C.: Characterizing microservice dependency and performance: Alibaba trace analysis. In: Proc. ACM SoCC. pp. 412–426 (2021)
8. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-refine: Iterative refinement with self-feedback. In: Proc. NeurIPS (2023)
9. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Proc. TACAS. pp. 337–340 (2008)
10. Shahrad, M., Fonseca, R., Goiri, Í., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., Bianchini, R.: Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In: Proc. USENIX ATC. pp. 205–218 (2020)
11. Sun, C., Sheng, Y., Padon, O., Barrett, C.: Clover: Closed-loop verifiable code generation. In: Proc. SAIV (2024)
12. Vardi, M.Y., Wolper, P.: An automata-theoretic approach to automatic program verification. In: Proc. LICS. pp. 332–344 (1986)
13. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. In: Proc. ICLR (2023)
14. Zeng, L., Benatallah, B., Ngu, A.H.H., Dumas, M., Kalagnanam, J., Chang, H.: QoS-aware middleware for web services composition. IEEE Trans. Softw. Eng. **30**(5), 311–327 (2004)
15. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.X.: Language agent tree search unifies reasoning, acting, and planning in language models. In: Proc. ICML (2024)